

System Architecture & Technical Reference

A complete technical overview of the Dentora platform — monorepo structure, backend API, AI/telephony pipeline, data layer, security model, infrastructure, and observability stack.

NEXT.JS 16 · APP ROUTER

EXPRESS 5 · DRIZZLE ORM

POSTGRESQL · REDIS · BULLMQ

TWILIO · OPENAI · ANTHROPIC

CLOUDFLARE R2 · DOCKER

OPENTELEMETRY · DATADOG

AES-256-GCM · UK GDPR

3
APP
WORKSPACES

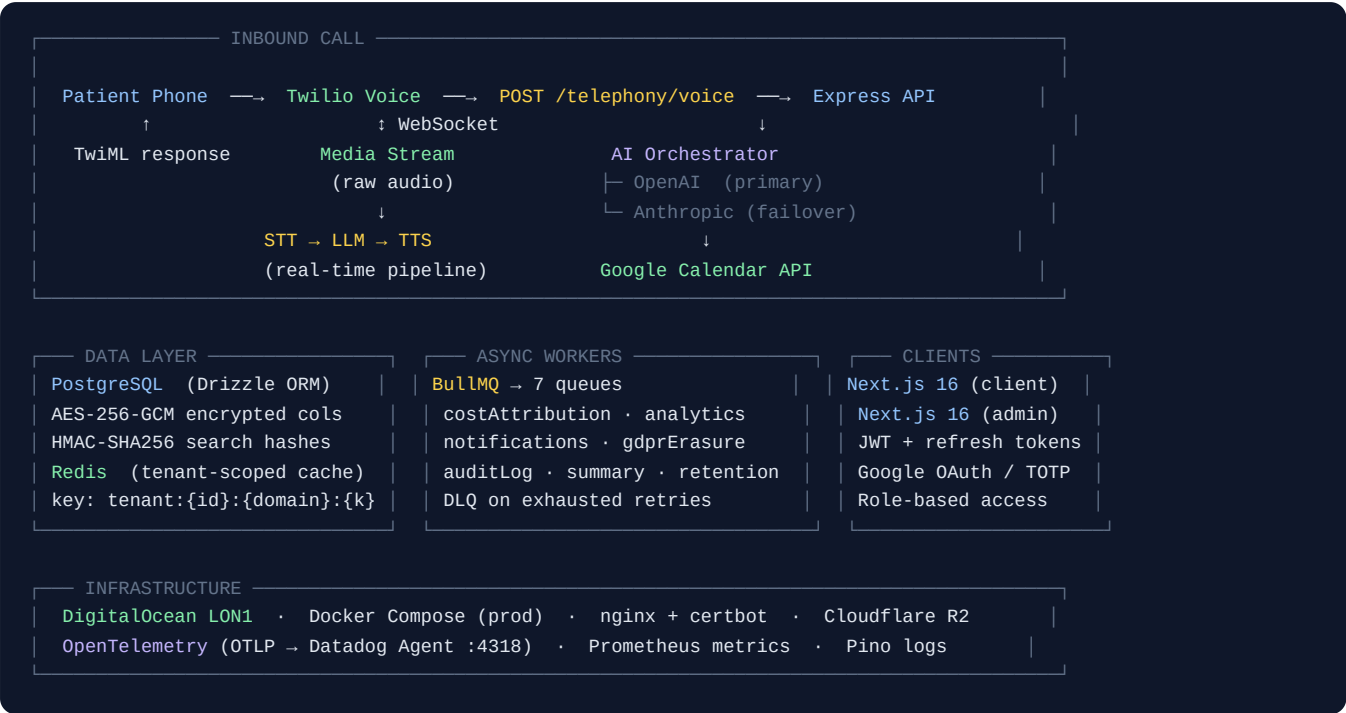
11
SERVER
MODULES

7
BULLMQ
QUEUES

256-bit
AES-GCM AT
REST

LON1
DIGITALOCEAN

1 HIGH-LEVEL SYSTEM ARCHITECTURE



2 MONOREPO STRUCTURE (PNPM WORKSPACES)

```
// pnpm workspace layout apps/ client/ - Next.js 16
App Router (patient dashboard) server/ - Express 5 API
+ BullMQ worker admin/ - Next.js 16 internal portal
(Biome lint) packages/ shared/ - Shared types +
constants // Target specific workspaces: pnpm --filter
@repo/server dev pnpm --filter @repo/client build pnpm
--filter @repo/shared typecheck
```

App	Framework	Linter	Output
client	Next.js 16 (App Router)	ESLint + Prettier	Vercel/Docker
server	Express 5 + tsc → CJS	ESLint	Node Docker
admin	Next.js 16 (App Router)	Biome	Vercel/Docker
shared	TypeScript lib	tsc only	workspace dep

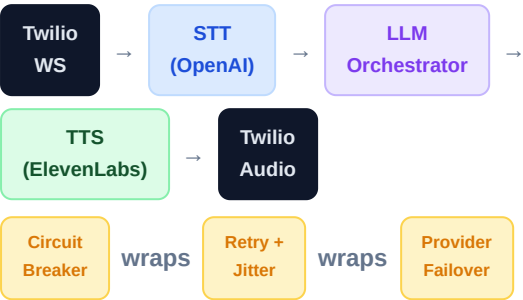
Note: The server compiles to CommonJS (`tsc --module commonjs`). `__dirname` is safe; `import.meta.url` is not available.

Server module map

Module	Responsibility
auth/	JWT, refresh tokens, Google OAuth, TOTP MFA, password reset
telephony/	Twilio webhooks, WS media stream, call lifecycle
ai/	LLM orchestration, provider failover, circuit breakers
patients/	PHI profiles (AES-encrypted), GDPR erasure, DSAR export
calls/	Call sessions, transcripts, events, cost attribution
config/	Clinic profile, services, booking rules, FAQs (Redis-cached)
integrations/	Google Calendar OAuth + live availability
appointments/	Booking logic, slot availability, conflict detection
analytics/	Call analytics, usage metrics, Datadog business events
tenants/	Tenant registry, plan management, isolation enforcement
staff-review/	Call review workflow, approval/rejection, notifications

3 AI & TELEPHONY PIPELINE

Call flow — media-stream.ts



LLM provider failover

```
// apps/server/src/lib/circuit-breaker.ts // 5-failure threshold → open → 30s reset
const cb = new CircuitBreaker({ threshold: 5, resetTimeout: 30_000 })
// Primary: OpenAI gpt-4o // Failover: Anthropic claude-sonnet-4-x // Wraps: all AI + Twilio external calls
```

Key lib files

File	Purpose
circuit-breaker.ts	5-failure threshold, 30s reset, wraps all external calls
retry.ts	Exponential backoff with jitter for transient failures
queue.ts	BullMQ — 7 queues, DLQ, OTEL spans on all jobs
cache.ts	tenantCacheGet/Set/Del, domain invalidation
metrics.ts	Prometheus metrics + Datadog business metrics
telemetry.ts	OTel SDK → OTLP → Datadog Agent :4318
logger.ts	Pino + dd.trace_id injection + PII redaction
storage.ts	Cloudflare R2 via aws4fetch (no AWS SDK)
encrypted-column.ts	AES-256-GCM encrypt/decrypt + HMAC-SHA256 hash

4 DATA LAYER

Schema highlights (schema.ts)

Table	Encrypted cols	Hash cols
patientProfiles	fullName, phoneNumber, notes	phoneNumberHash
callSessions	callerNumber, intentSummary	callerNumberHash
callTranscripts	summary	—
auditLog	—	—
passwordResetTokens	—	tokenHash

BullMQ queue definitions

COST_ATTRIBUTION

ANALYTICS_EVENTS

NOTIFICATION_DELIVERY

GDPR_ERASURE

AUDIT_LOG_WRITE

CALL_SUMMARY_GEN

DATA_RETENTION_SWEEP

Cache key convention

```
// Tenant-scoped Redis keys tenant:{tenantId}:{domain}:{id} // Config domain — 5 min TTL
tenant:abc123:config:clinic // Invalidate after config write: tenantCacheInvalidateDomain(tenantId, 'config')
```

Tenant isolation rule

Every DB query must include `eq(table.tenantId, tenantId)`. `tenantId` is always sourced from JWT (`req.tenantContext.tenantId`) — never from request body. Enforced by `resolveTenant` middleware.

Migration workflow

```
1. Edit src/db/schema.ts 2. pnpm db:generate -- drizzle-kit 3. Review drizzle/*.sql 4. pnpm db:migrate -- local apply // NEVER use drizzle-kit push in prod // Migrations run at deploy via migrate cmd
```

5 SECURITY ARCHITECTURE

Encryption at rest — AES-256-GCM

```
// src/lib/encrypted-column.ts export
encryptField(plaintext: string): string // AES-256-
GCM: 96-bit IV per encrypt // Format: base64(iv ||
ciphertext || authTag) export decryptField(ciphertext:
string): string export hashForSearch(plaintext:
string): string // HMAC-SHA256 — deterministic, non-
reversible // Used for phone number lookup without
decrypt
```

Protected field	Method	Searchable?
Patient full name	AES-256-GCM	No
Patient phone number	AES-256-GCM + HMAC hash	Yes (hash lookup)
Patient notes	AES-256-GCM	No
Caller number (call)	AES-256-GCM + HMAC hash	Yes (hash lookup)
Intent summary	AES-256-GCM	No
Transcript summary	AES-256-GCM	No

Transport security

- TLS 1.3 enforced on all endpoints via nginx
- Let's Encrypt certificates (auto-renewed via certbot container)
- WebSocket connections (Twilio media stream) over WSS
- Strict CORS — origin whitelist only, no wildcard
- HSTS, X-Frame-Options, X-Content-Type-Options headers set

UK GDPR technical controls

Article	Implementation
Art. 15 (Access)	GET /api/patients/:id/export → full structured export
Art. 17 (Erasure)	DELETE /api/patients/:id → cascade deletion via patient.deletion.ts
Art. 20 (Portability)	JSON export via same endpoint, machine-readable format
Art. 25 (Privacy by design)	Encryption, minimal collection, per-tenant isolation in schema
Art. 30 (Processing records)	Audit log: every req.audit?.() call, 7-year retention

6 AUTHENTICATION & MIDDLEWARE STACK

Auth flow

1. Login: email+password → bcrypt verify
2. TOTP MFA check (authenticator app)
3. Issue: JWT (access, 15 min) + refresh token (httpOnly cookie)
4. Every API request: authenticateJwt → resolveTenant
5. req.user: { userId, tenantId, role } — no email in token

JWT payload

```
// req.user injected by authenticateJwt middleware {
userId: "uuid", tenantId: "uuid", // NOT from body
role: "admin" | "staff" } // email is NOT in the token
// fetch via authService.getUserAccountInfo()
```

Middleware chain (all protected routes)

```
authenticateJwt // verify + decode JWT resolveTenant
// pin tenantId from token rateLimiter // per-IP +
per-tenant limits validate({body}) // Zod schema
validation routeHandler // business logic req.audit?.
() // append to audit log
```

Rate limiting

Module	Limit	Window
patients	120 req	60 s
auth (login)	10 req	60 s
general API	300 req	60 s

Audit log is append-only. Every call to req.audit?.() writes a timestamped record with action, entityType, userId, and tenantId. Retained 7 years; no UPDATE or DELETE is ever issued against this table.

7

INFRASTRUCTURE & DEPLOYMENT

Production Docker Compose services

Service	Image / Role
postgres	PostgreSQL 16 — primary database
redis	Redis 7 — cache + BullMQ broker
server	Express 5 API (Node 22-alpine)
worker	BullMQ worker process (separate container)
client	Next.js 16 patient dashboard
nginx	Reverse proxy + TLS termination
certbot	Let's Encrypt certificate auto-renewal
datadog-agent	OTel collector + APM agent

Worker isolation: the worker container runs `apps/server/src/worker.ts` as a separate process. Worker code must never be imported into the API server container — the boundary is enforced at the Docker service level.

Deploy pipeline

```
# CI: TypeScript + tests + lint must pass # On master
push → deploy.yml GitHub Action git pull origin master
docker compose -f docker-compose.prod.yaml pull docker
compose -f docker-compose.prod.yaml \ up -d --remove-orphans
# Migrations run automatically at server start
```

File storage — Cloudflare R2

```
// src/lib/storage.ts (aws4fetch, no AWS SDK)
uploadFile // context docs, call recordings
downloadFile // retrieve stored objects getSignedUrl
// time-limited presigned URLs deleteFile // called on
GDPR erasure // Env: R2_BUCKET, R2_ACCOUNT_ID, //
R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY
```

8

OBSERVABILITY STACK

Tracing

OpenTelemetry SDK → OTLP → Datadog Agent (:4318). All BullMQ jobs carry OTEL spans. Twilio + AI calls traced end-to-end.

Metrics

Prometheus endpoint + Datadog custom metrics. Business events (bookings, escalations, costs) tracked as distribution metrics.

Logging

Pino JSON logger. `dd.trace_id / dd.span_id` injected into every log line. PII fields redacted before emission.

Health checks

GET `/health` reports DB, Redis, worker, and media-stream status. Used by Docker Compose `healthcheck` and nginx upstream probe.

9

CORE API REFERENCE

Endpoint	Auth	Purpose
POST <code>/auth/login</code>	—	Email+password → JWT + refresh cookie
POST <code>/auth/forgot-password</code>	—	Send 1-hour reset token via email
POST <code>/auth/reset-password</code>	—	Consume reset token, set new password
GET <code>/api/patients</code>	JWT	List patients (tenantId scoped)
POST <code>/api/patients/lookup</code>	JWT	Find by phone+DOB (HMAC hash lookup)
GET <code>/api/patients/:id/export</code>	JWT	GDPR Art. 15/20 data export
DELETE <code>/api/patients/:id</code>	JWT	GDPR Art. 17 cascade erasure
GET <code>/api/calls</code>	JWT	List call sessions (cursor pagination)
POST <code>/telephony/voice</code>	Twilio sig	Incoming call webhook → TwiML response
WS <code>/telephony/stream</code>	Twilio sig	Real-time audio media stream
GET <code>/api/config</code>	JWT	Clinic config (Redis-cached, 5 min)
PUT <code>/api/config</code>	JWT	Update config → invalidate cache
GET <code>/api/analytics</code>	JWT	Call analytics +